UniMarket

Running the full platform on your own machine

LOCAL DEVELOPMENT GUIDE

Laravel 13 · Livewire 3 · Filament 3 · MySQL (XAMPP)

What "fully running" means

Five things need to be active at the same time for every feature to work - the database, the web server, the asset builder, the queue worker, and the scheduler. Missing any one of them doesn't crash the site, it just quietly breaks one part of it (e.g. no scheduler means verification reminder emails never go out).

Service	What it does	What breaks without it
MySQL (XAMPP)	Stores all app data	Nothing works at all - the site won't load
Web server `php artisan serve`	Serves the site itself	Nothing works at all - the site won't load
Vite dev server `npm run dev`	Compiles CSS/JS, hot-reloads changes	Pages show unstyled/broken, or a "Vite manifest" error
Queue worker `php artisan queue:work`	Delivers queued notifications	Price-drop alerts, saved-search matches and new-listing alerts never arrive
Scheduler `php artisan schedule:work`	Runs the daily/hourly background jobs	Verification reminders, the 30-day document purge, insights pruning, etc. never run

> This project's own AGENTS.md file assumes Laravel Herd serves the site automatically. On this machine Herd isn't actually installed - MySQL is XAMPP's, and the site runs via `php artisan serve` instead. This guide reflects what's actually on this machine, not that assumption.

Fastest way: the startup script

A ready-made script already checks and starts everything in one go. It's safe to run any time - it skips anything that's already running instead of starting a duplicate.

```
cd "C:\Users\PERSIS PELEKEL0\Desktop\uni-market"
powershell -ExecutionPolicy Bypass -File .\scripts\start-platform.ps1
```

Or, from inside PowerShell already in the project folder:

```
.\scripts\start-platform.ps1
```

Each service runs quietly in the background - no windows to manage - and prints a status line for each:

```
== UniMarket platform startup ==
[OK] MySQL already running
[OK] Web server already running on http://127.0.0.1:8000
[OK] Vite dev server already running
[OK] Queue worker already running
[OK] Scheduler already running

== Done ==
App:        http://127.0.0.1:8000
Admin:      http://127.0.0.1:8000/admin
Logs:       storage\logs\*.out.log / *.err.log - one pair per service
```

> Each service writes its own output to `storage\logs\<service>.out.log` and `.err.log` (server, vite, queue, schedule) - check those if something seems off. To stop everything, find the processes in Task Manager (look for `php.exe` and `node.exe`) and end them, or close the PowerShell window the script itself ran in if it's still open.

> The script also checks `public\storage` on every run and automatically fixes it if it was ever recreated from Git Bash instead of PowerShell - see the troubleshooting table for why that matters.

Doing it by hand instead

If you ever need to start things manually (or the script can't run for some reason), open **four separate terminal windows** in the project folder and run one command in each. Leave all four running.

1. **Start MySQL** - open the XAMPP Control Panel and click Start next to MySQL. (Or run `C:\xampp\mysql_start.bat`.)

2. **Start the web server** (Terminal 1):

   ```
   cd "C:\Users\PERSIS PELEKEL0\Desktop\uni-market"
   php artisan serve
   ```

   Leave it running. The site is now at `http://127.0.0.1:8000`.

3. **Start the asset builder** (Terminal 2):

   ```
   cd "C:\Users\PERSIS PELEKEL0\Desktop\uni-market"
   npm run dev
   ```

4. **Start the queue worker** (Terminal 3):

   ```
   cd "C:\Users\PERSIS PELEKEL0\Desktop\uni-market"
   php artisan queue:work --tries=1
   ```

5. **Start the scheduler** (Terminal 4):

   ```
   cd "C:\Users\PERSIS PELEKEL0\Desktop\uni-market"
   php artisan schedule:work
   ```

First time only: setting it up from scratch

Skip this section if the project already has a working `.env` and database. Only needed on a brand-new checkout.

1. Install PHP dependencies:

```
composer install
```

2. Create the environment file and app key:

```
copy .env.example .env
php artisan key:generate
```

   Then open `.env` and check the database settings match a MySQL database you've created (`DB_DATABASE` , `DB_USERNAME` , `DB_PASSWORD`).

3. Create the database tables:

```
php artisan migrate
```

4. Link the storage folder (needed for uploaded photos/avatars):

```
php artisan storage:link
```

5. Install and build frontend assets:

```
npm install
npm run build
```

6. Optional - seed demo data and test accounts (local environment only):

```
php artisan demo:reset
```

   This wipes and recreates the database with the three accounts listed below.

Checking everything actually works

- Visit **http://127.0.0.1:8000** - the marketplace home page should load with styling (if it looks unstyled, Vite isn't running - see step 3 above).
- Visit **http://127.0.0.1:8000/admin** and sign in with the admin account below - you should reach the Filament dashboard.
- Run `php artisan pail` in any terminal to tail the application log live - since this project sends email through the `log` driver, every "sent" email shows up here instead of a real inbox.
- Run `php artisan queue:monitor database` (or just watch the queue worker's terminal) to confirm queued notifications are being picked up.

Demo / test accounts

Created by `php artisan demo:reset` (local environment only):

Role	Email	Password
Admin	admin@zut.zm	Admin@2026
Seller (verified student)	seller@student.zut.zm	Seller@2026
Buyer (verified student)	buyer@student.zut.zm	Buyer@2026

Troubleshooting

Symptom	Likely cause	Fix
Blank page or "Connection refused"	MySQL isn't running	Open XAMPP Control Panel, start MySQL
"Vite manifest" error	Vite dev server isn't running, and no production build exists either	Run `npm run dev` (or `npm run build` for a one-off static build)
Page loads but looks unstyled	Same as above	Same as above
Notification emails never "arrive"	They're not queued - check `php artisan pail` instead of an inbox, since `MAIL_MAILER=log`	This is expected in local dev, not a bug
Saved-search / price-drop alerts never fire	Queue worker or scheduler isn't running	Start both (steps 4-5 above, or re-run the script)
Two web server processes, port conflict	An old server process was never closed	End every `php.exe` process in Task Manager, then re-run the script
Every photo/avatar/document request hangs for ~30 seconds, then fails	`public\storage` is a broken symlink - it happens if that link ever gets recreated from Git Bash's `ln -s` instead of PowerShell, since it then points to a Unix-style path (`/c/Users/...`) that native Windows PHP can't resolve	The script fixes this automatically on every run; to fix it by hand, delete `public\storage` and run `php artisan storage:link --force` from PowerShell (not Git Bash)
"php artisan serve" itself hangs with no output or error	Its process-monitoring wrapper misbehaves on this machine for reasons unrelated to the app	Not a bug in the app - the script avoids this entirely by running the same underlying command (`php -S`) directly instead

Generated for the UniMarket project. The startup script lives at `scripts\start-platform.ps1` in the project folder.